Day 3: NLP, Encoder-Decoder & Transformers

Comprehensive Study Handbook: Intuitive Concepts, Mathematics & Architectures

1. Natural Language Processing (NLP) & The Evolution of Language Models

• **What is NLP?** Teaching computers to understand, interpret, and generate messy, contextual human language.
• **Why is it Hard?** Ambiguity (*'river bank'* vs *'money bank'*), sarcasm, syntax vs. semantics (*'Colorless green ideas sleep furiously'* is grammatically valid but semantically meaningless).
• **The 4 Eras:** 1. Rule-Based (Regex/N-grams) → 2. Statistical (TF-IDF/Naive Bayes) → 3. Recurrent (Word2Vec, RNN/LSTM) → 4. Transformer Era (2017–Present: Attention, BERT, GPT, Gemini).

2. Text Tokenization: How Computers Chop Text into IDs

Level	How It Works	Example: 'unhappy'	Pros & Cons
Character	Every letter/symbol is a token	`['u','n','h','a','p','p','y']`	No OOV words; but long sequences and letters lack meaning.
Word	Split by whitespace/punctuation	`['unhappy']`	Direct meaning; but giant vocabulary (>500k) and fails on typos.
Subword (BPE)	Frequent words whole; rare split into morphemes	`['un', 'happy']`	**Modern Standard (GPT, BERT)!** Compact vocab (~50k), handles any word.

3. Dense Word Embeddings: The Geometry of Meaning

• **Why Not One-Hot Encoding?** One-hot vectors are huge (50,000 dims) and completely sparse. More critically, the dot product between any two one-hot vectors is **0.0** (no concept of semantic similarity!).
• **Dense Embeddings:** Map words into compact continuous vectors (e.g., 768 dims). Words with similar meanings cluster together in vector space.
• **Semantic Vector Arithmetic:** *vector('King') – vector('Man') + vector('Woman') ≈ vector('Queen')*.
• **Cosine Similarity:** Measures angle between vectors: $\cos(\theta) = (u \cdot v) / (||u|| \cdot ||v||)$. Values range from –1.0 (opposite) to +1.0 (identical direction).

Day 3: Scaled Dot-Product & Self-Attention

Breaking the Sequential Bottleneck: From RNNs to Attention

4. The RNN/LSTM Bottleneck: Why Transformers Won

• **Sequential Execution:** RNNs process tokens one by one (word 1 → word 2 → word 3). Modern GPUs have thousands of cores, but RNNs forced them to wait sequentially.
• **Information Forgetting:** RNNs cram the entire sentence history into a single hidden vector h_t. By token 50, early context is largely lost to vanishing gradients.
• **The Attention Revelation (Vaswani et al., 2017):** *'Attention Is All You Need'*. Remove recurrence completely! Ingest all words in parallel, and let every word directly attend to every other word.

5. The Library Analogy: Query, Key, and Value

Component	Library Analogy	Mathematical Role in Self-Attention
Query (Q)	The search question you bring to the librarian	Current token asking: *'What information do I need from others?'*
Key (K)	The title/index tag on book spines	Other tokens advertising: *'Here is what I represent!'*
Value (V)	The actual knowledge inside the book pages	The content vector that gets extracted and blended into the output.

6. Scaled Dot-Product Attention: Step-by-Step Math

The Golden Formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

• **Step 1 (Raw Scores):** Compute dot product $Q \cdot K^T$. High dot product = high relevance between tokens.
• **Step 2 (Scaling by $\sqrt{d_k}$):** If d_k is large (e.g. 64), dot products explode in magnitude. Large values drive Softmax into flat saturated zones where gradients vanish (become 0)! Dividing by $\sqrt{d_k}$ maintains unit variance and healthy gradients.
• **Step 3 (Softmax):** Normalizes scaled scores into probabilities summing to exactly 1.0 (100%).
• **Step 4 (Weighted Sum):** Multiply attention probabilities by V to produce a context-rich blended output vector for each word.
• **Multi-Head Attention (MHA):** Splits the embedding into h heads (e.g., 8 or 16). Head 1 tracks grammar (subject-verb), Head 2 tracks pronoun resolution ('it' → 'animal'), Head 3 tracks themes.

Day 3: Transformer Architecture & Positional Encodings

Building Deep, Scalable Language Models

7. The Permutation Invariance Problem & Positional Encoding

• **The Problem:** Self-attention computes pairwise dot products simultaneously across all tokens. Without order information, *'Dog bites man'* and *'Man bites dog'* produce identical attention scores! Self-attention is order-agnostic.

• **The Sinusoidal Solution (Vaswani et al.):** Add a unique positional wave vector to each token's embedding:

$$PE_{(pos,\ 2i)} = sin(\ pos\ /\ 10000^{2i/d_{model}}\ ) \quad | \quad PE_{(pos,\ 2i+1)} = cos(\ pos\ /\ 10000^{2i/d_{model}}\ )$$

• **Why Sine & Cosine?** Low dimensions oscillate rapidly (like a seconds hand, tracking immediate neighbors); high dimensions oscillate slowly (like an hour hand, tracking global sentence position). Enables models to easily learn **relative positions** via linear transformations.

8. The 4 Essential Components Inside Every Transformer Block

Component	Mechanism & Formula	Why It Is Crucial
Multi-Head Attention	$Concat(head_1, ..., head_h) \cdot W_O$	Allows tokens to gather context across multiple linguistic relationships simultaneously.
Residual Connection	$Output = x + Sublayer(x)$	Solves vanishing gradients: creates an uninterrupted gradient highway through 100+ layers.
Layer Normalization	Normalize features for a single token: $\mu=0,\ \sigma^2=1$	Unlike BatchNorm, LayerNorm does not depend on batch size or sequence length.
Feed-Forward Net (FFN)	$Linear \rightarrow GELU \rightarrow Linear$ (expands 4× dim)	Acts as the permanent key-value memory store of factual world knowledge in LLMs.

9. Cross-Attention: How the Decoder Listens to the Encoder

In Encoder-Decoder models (like T5):

• **Queries (Q):** Generated by the **Decoder** (the target sentence being written, e.g. German).

• **Keys (K) & Values (V):** Provided by the **Encoder** output (the source sentence, e.g. English).

• This allows the decoder to reference the source text at every generation step!

Day 3: BERT, GPT, T5 & LLM Text Generation

The Transformer Trinity & Autoregressive Decoding Strategies

10. The Great Trinity: BERT vs. GPT vs. T5

Architecture	Attention Mask	Pre-Training Task	Best For & Examples
Encoder-Only (BERT)	**Bidirectional** (Full 1s matrix: sees left & right)	**Masked LM (MLM):** *'The [MASK] sat on mat'*	Classification, NER, Search, Sentiment (BERT, RoBERTa, DeBERTa)
Decoder-Only (GPT)	**Causal Mask** (Lower triangular: sees past only)	**Next-Token Prediction:** $P(token_{t+1} \mid token_{1...t})$	Text generation, Chat, Coding, Reasoning (GPT-4, LLaMA, Gemini)
Encoder-Decoder (T5)	Enc: Bidirectional Dec: Causal + Cross-Attn	**Text-to-Text:** Span corruption & reconstruction	Translation, Abstractive Summarization (T5, FLAN-T5, BART)

11. How LLMs Generate Text & Decoding Strategies

• **Autoregressive Loop:** 1. Encode prompt → 2. Forward pass → 3. Extract logits of final token → 4. Scale by Temperature (z_i/T) → 5. Softmax to probabilities → 6. Sample next token → 7. Append and repeat!
• **Greedy Search (argmax):** Always picks top-1 token. Fast, deterministic, but repetitive.
• **Temperature (T):** $T < 1.0$ sharpens distribution (logical, coding, math); $T > 1.0$ flattens it (creative, diverse).
• **Top-K Sampling:** Limits choices to top K tokens (e.g. 50), eliminating strange low-probability words.
• **Top-P (Nucleus) Sampling: Industry Standard!** Dynamically keeps the smallest set of tokens whose cumulative probability ≥ P (e.g. 90%). Adaptively expands when uncertain and contracts when confident.

12. High-Yield Interview Flashcards

• **Q: What is the computational complexity of standard self-attention?**
A: $O(N^2)$ in time and memory, where N is sequence length, because every token attends to all N tokens.
• **Q: Why do LLMs make silly math errors (e.g., 98765 + 4321)?**
A: Tokenizer artifacts! Multi-digit numbers are chopped into arbitrary subword chunks (e.g. ['987', '65']), obscuring place value alignment without step-by-step scratchpads.
• **Q: Why does GPT need a Causal Mask?**
A: To prevent the model from looking ahead at future tokens during training, which would allow it to cheat instead of learning to predict.